

基于 Python 的 LLM Agent

设计与实现

—— MyAgent 项目学习报告

课程名称：人工智能科学与技术

项目名称：MyAgent——LLM Agent 学习项目

姓 名：沈一村

学 号：2550285

专业：工科试验班(信息与智能网联类)

日 期：2026 年 6 月 1 日

目录

第一章 项目概述	4
1.1 项目背景	4
1.2 项目目标	4
1.3 技术栈	4
1.4 项目结构	5
第二章 版本迭代：从 V1 到 V6	7
2.1 V1：单次对话（~25 行）	7
核心代码	7
学到的知识	7
2.2 V2：循环多轮对话（~90 行）	8
V1→V2 关键改动	8
核心概念：对话记忆 = 一个不断增长的列表	9
学到的知识	9
2.3 V3：工具调用（~140 行）	10
V2→V3 关键改动	10
工具调用的完整流程	11
踩过的坑	11
2.4 V4：配置化 + 工具生态（~180 行）	12
V3→V4 关键改动	12
设计亮点：配置驱动的工具注册	13
踩过的坑	13
2.5 V5：记忆系统（~350 行）	15
V4→V5 关键改动	15
记忆 ≠ 对话历史	15

重大技术挑战: XML 工具调用幻觉	16
2.6 V6: 多会话管理 + 对话压缩 (~590 行)	17
V5→V6 核心改动	17
踩过的坑	18
第三章 核心功能详解	20
3.1 工具系统架构	20
工具进化史	21
3.2 记忆系统原理	21
3.3 会话管理设计	22
3.4 对话压缩算法	22
第四章 关键技术问题与解决方案	24
4.1 XML 工具调用幻觉	24
4.2 reasoning_content 400 错误	24
4.3 round_num 计数 Bug	24
4.4 全量合并历史的性能问题	25
第五章 项目成果展示	26
5.1 系统命令一览	26
5.2 工具能力展示	26
第六章 总结与展望	28
6.1 项目总结	28
6.2 技术亮点	28
6.3 不足与改进方向	28
6.4 学习心得	29
参考文献	29

第一章 项目概述

1.1 项目背景

随着大语言模型（LLM，Large Language Model）技术的飞速发展，AI Agent（人工智能代理）已成为人工智能领域的重要研究方向。传统的 LLM 聊天机器人只能进行文本对话，而 AI Agent 则能够调用外部工具（如搜索引擎、计算器、文件系统等），自主完成复杂任务。本项目旨在从零开始，使用 Python 语言构建一个具备工具调用、记忆管理、多会话管理等能力的 LLM Agent，深入理解 AI Agent 的核心原理与实现方式。整个项目，包含项目报告，均在 LLM Agent 的协助下完成。

本项目 GitHub 仓库：<https://github.com/LiuAn-HuaMing/MyAgent>

1.2 项目目标

- 掌握 OpenAI SDK 的使用方法，实现与 DeepSeek 大语言模型的交互
- 实现 Agent 工具调用机制（Function Calling），使模型能够调用外部工具
- 构建跨会话记忆系统，使 Agent 能够持久化用户偏好和重要信息
- 实现多会话管理，支持会话存档、切换、删除
- 实现对话压缩功能，在长对话中自动摘要旧消息以节省 Token 消耗
- 通过 V1→V6 的迭代开发，记录完整的 Agent 演进过程

1.3 技术栈

技术/工具	版本/说明	用途
Python	3.13	主编程语言
OpenAI SDK	>=1.0.0	调用 DeepSeek API（兼容 OpenAI 格式）
DeepSeek V4	deepseek-v4-pro	大语言模型（推理模式）
python-dotenv	>=1.0.0	.env 环境变量管理，保护 API 密钥

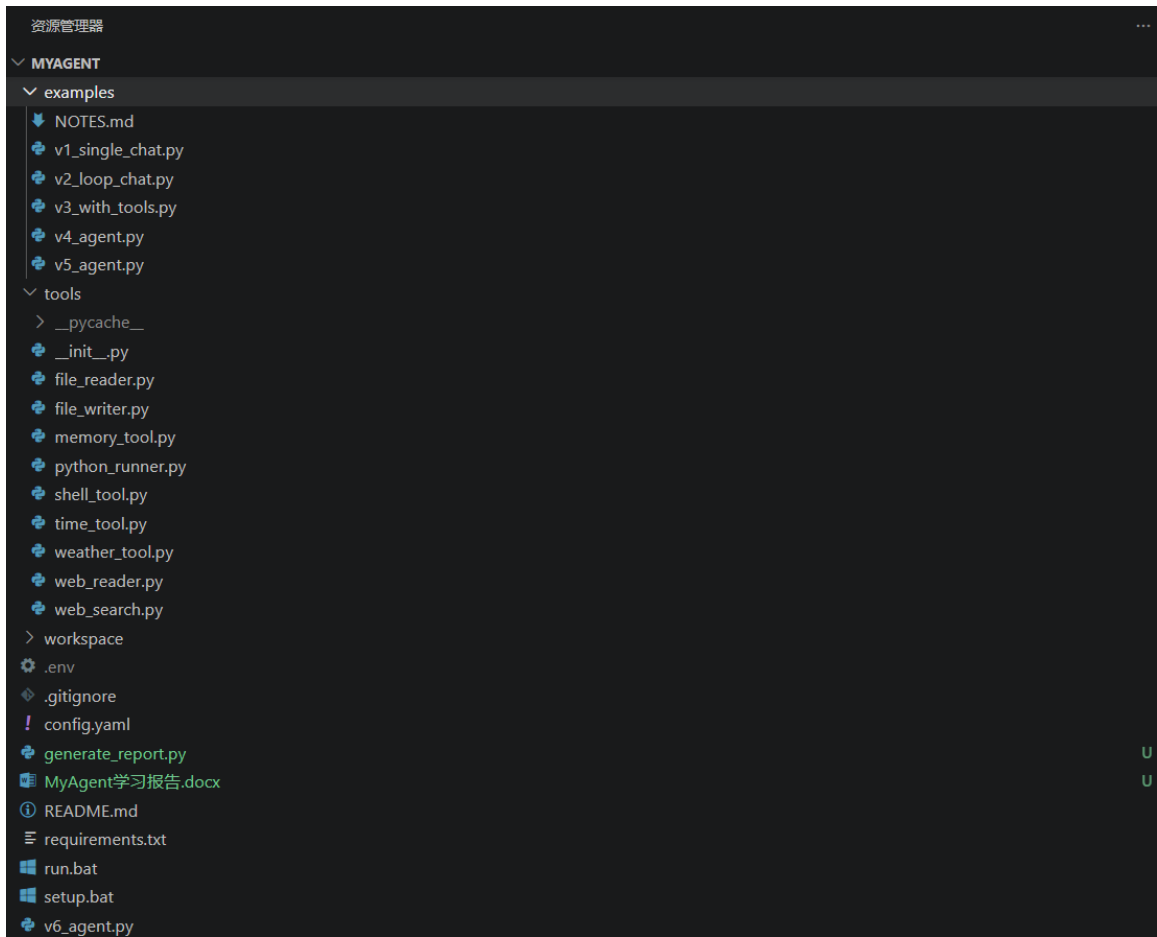
PyYAML	>=6.0	解析 config.yaml 配置文件
BeautifulSoup4	>=4.0	网页内容解析
Colorama	>=0.4	终端彩色输出

1.4 项目结构

项目采用模块化设计，各组件职责清晰：

```
MyAgent/
├─ v6_agent.py           # 主程序（当前版本，~590 行）
├─ config.yaml          # 配置文件（模型参数、工具开关、压缩设置）
├─ requirements.txt     # Python 依赖包列表
├─ .env                 # API 密钥（不提交到 Git）
├─ README.md           # 项目说明文档
├─ tools/              # 工具包（11 个工具）
│   ├── __init__.py     # 工具注册中心
│   ├── time_tool.py    # 时间查询
│   ├── weather_tool.py # 天气查询（wttr.in）
│   ├── file_reader.py  # 文件读取
│   ├── file_writer.py  # 文件写入（沙箱模式）
│   ├── web_search.py   # 网页搜索（Bing）
│   ├── web_reader.py   # 网页全文阅读
│   ├── shell_tool.py   # 命令行执行（需确认）
│   ├── python_runner.py # Python 代码执行（沙箱）
│   └─ memory_tool.py   # 记忆管理（增删查）
├─ examples/           # V1~V5 版本示例 + 学习笔记
│   ├── NOTES.md        # 完整学习笔记（本文档重要参考）
│   ├── v1_single_chat.py # V1: 单次对话
│   ├── v2_loop_chat.py  # V2: 循环多轮对话
│   ├── v3_with_tools.py # V3: 工具调用
│   ├── v4_agent.py      # V4: 配置化 + 工具生态
│   └─ v5_agent.py       # V5: 记忆系统
└─ workspace/          # 运行时工作目录（不提交）
    └─ conversations/   # 会话存档（独立 JSON 文件）
```

└─ memory.json # 跨会话记忆持久化
└─ (Agent 创建的文件)



项目文件夹结构截图

第二章 版本迭代：从 V1 到 V6

本项目采用迭代开发模式，每个版本在上一版本的基础上增加新功能，逐步构建完整的 Agent 系统。下面详细描述每个版本的目标、核心改动、关键代码片段和学到的知识。

2.1 V1：单次对话（~25 行）

目标：跑通 DeepSeek API，理解最基本的“发消息→收回复”流程。

核心代码

```
# V1 核心逻辑（简化）
from openai import OpenAI
from dotenv import load_dotenv
import os

load_dotenv()
client = OpenAI(
    api_key=os.getenv("DEEPSEEK_API_KEY"),
    base_url=os.getenv("DEEPSEEK_BASE_URL"),
)

question = input("IN [1]: ")
response = client.chat.completions.create(
    model=os.getenv("DEEPSEEK_MODEL"),
    messages=[{"role": "user", "content": question}]
)
print(f"OUT[1]: {response.choices[0].message.content}")
```

学到的知识

- OpenAI SDK 的 `client.chat.completions.create()` 是真正发送 HTTP 请求的方法
- `messages` 列表里每条消息有 `role` (user/assistant/system) 和 `content` 两个字段
- `response.choices[0].message.content` 用于提取模型的回复文本
- `.env` 文件 + `python-dotenv` 是最佳的密钥管理方案
- DeepSeek API 完全兼容 OpenAI SDK 接口格式，切换成本极低

```
PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/examples/v1_single_chat.py
=====
👤 对话助手
=====

IN [1]: 你好
OUT[1]: 你好! 🌟 我是DeepSeek, 很高兴见到你!

我可以帮你解答问题、提供建议、进行讨论, 或者纯粹陪你聊聊天。无论是严肃的知识问题, 还是轻松的日常话题, 我都很乐意参与。

今天有什么我可以帮你的吗? 尽管说, 不用客气! 😊
PS C:\Programming\MyAgent> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:/Programming/MyAgent/.venv/Scripts/Activate.ps1)
(.venv) PS C:\Programming\MyAgent> |
```

V1 运行截图：单次问答示例

2.2 V2：循环多轮对话（~90 行）

目标：像网页版 ChatGPT 一样持续聊天，能记住上下文。

V1→V2 关键改动

改动项	说明
while True 循环	替代一次性执行，实现持续对话
messages 列表记忆	将全部历史消息存在列表中，每轮全量发送给模型——这是“记忆”的最简单形态
流式输出（stream=True）	实时显示模型的思考和回答过程，改善等待体验
系统命令	/exit、/clear、/help、/stats 四类命令，扩展了终端交互能力
Token 估算统计	中文字符 ×0.6 + 英文 ×0.3 的粗略估算，帮助了解用量

核心概念：对话记忆 = 一个不断增长的列表

```
# V2 的记忆实现（最简形态）

messages = [{"role": "system", "content": "你是一个友好的 AI 助手"}]

while True:
    user_input = input("...")
    messages.append({"role": "user", "content": user_input}) # 追加用户消息

    response = client.chat.completions.create(
        model=model,
        messages=messages, # 全量发送历史消息
        stream=True,
    )
    # ... 收集回答 ...
    messages.append({"role": "assistant", "content": answer}) # 追加模型回复
```

学到的知识

- DeepSeek 支持前缀缓存（Prefix Caching），重复的历史部分只需约 1/10 价格
- f-string (f"... {var}...") 比加号拼接更安全、更易读
- 流式输出大幅改善用户体验，思考过程实时可见

```

=====
🗨️ 对话助手 (输入 /help 查看命令)
=====

IN [1]: 你好?

THINK : 我们被问到“你好?”。这是一个简单的问候。我需要用中文回应,友好地打招呼。目前没有更多上下文。我可以简单回应“你好!有什么可以帮你的吗?”或者类似的话。
OUT[1]: 你好!很高兴见到你,有什么可以帮你的吗?
[本轮: 入~1t, 出~11t | 累计: ~22t]

IN [2]: 讲个简短的小故事吧

THINK : 好的,用户让我讲个简短的小故事。从之前的对话看,我们刚刚打过招呼,这是第一次提出具体请求。用户可能只是想在轻松的氛围中听个有趣的故事,消遣一下,或者测试一下我的能力。

我需要选一个真正简短、有寓意又适合口头讲述的故事。太复杂或太长就不符合“简短”的要求了。想到了“父子骑驴”这个经典寓言,它结构简单,情节有反复,最后有明确的道理,很适合。可以讲得生动一点,把路人的反应和父子俩的调整过程描述清楚。

讲完后,考虑到用户可能喜欢这类内容,可以主动问一下是否还想听别的,保持互动性。
OUT[2]: 当然可以。这是一个简短的小故事:

**《父子和驴》**

从前,有对父子赶着一头驴去集市。

一开始,儿子骑驴,父亲走路。路人见了纷纷议论:“这孩子真不懂事,让老父亲走路,自己却骑驴!”儿子听了羞愧,赶紧下来,让父亲骑上去。

可没走多远,又有路人指点:“这父亲怎么忍心,自己骑驴,让年幼的孩子走路?”父亲一听也觉得不妥,于是两人一起骑上了驴。

这时,一个老人路过,叹气道:“你们俩这么重,驴怎么受得了?这是虐待牲口啊!”

父子俩不知所措,最后只好抬着驴走。结果走到桥中间,驴挣扎了一下,掉进了河里,被水冲走了。

**寓意**: 一味听从别人的意见,最后可能既丢了驴,也失了主意。
[本轮: 入~5t, 出~155t | 累计: ~182t]

IN [3]: /exit
👋 再见!
PS C:\Programming\MyAgent> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Programming\My

```

运行截图: 多轮对话 + 思考过程展示

2.3 V3 : 工具调用 (~140 行)

目标: Agent 不仅能聊天,还能“做事”——调用计算器、查询时间。这是 Agent 区别于普通聊天机器人的核心能力。

V2→V3 关键改动

- 创建 tools/ 包: 每个工具一个 Python 文件,包含实现函数和 JSON Schema 描述
- tools/__init__.py 注册中心: 统一管理 TOOL_FUNCTIONS 字典和 TOOL_DESCRIPTIONS 列表

- API 调用加入 `tools=TOOL_DESCRIPTIONS` 参数，告知模型有哪些工具可用
- 检查 `msg.tool_calls` 字段：若不为空则执行对应工具函数
- 工具调用和结果用 `role: "tool" + tool_call_id` 加入消息历史
- `while True` 循环处理多次连续工具调用（模型可能一轮调用多个工具或多轮调用）

工具调用的完整流程

```
# V3 工具调用循环（简化）
while True:
    response = client.chat.completions.create(
        model=model,
        messages=messages,
        tools=TOOL_DESCRIPTIONS, # 关键：告诉模型有哪些工具
    )
    msg = response.choices[0].message

    if not msg.tool_calls: # 不需要工具 → 退出循环，直接回答
        final_msg = msg
        break

    # 执行每个工具调用
    for tool_call in msg.tool_calls:
        tool_func = TOOL_FUNCTIONS[tool_call.function.name]
        tool_result = tool_func(**json.loads(tool_call.function.arguments))

    # 将工具调用和结果加入历史
    messages.append({"role": "assistant", "tool_calls": msg.tool_calls, ...})
    messages.append({"role": "tool", "tool_call_id": ..., "content": tool_result})
```

踩过的坑

- Python 的 `^` 运算符是异或（XOR），不是幂运算——数学表达式中需转换为 `**`
- DeepSeek 推理模式下，`reasoning_content` 必须原样传回，否则下次 API 调用报 400 错误（KV-cache 无法命中）
- 模型可能连续调用多次工具，必须用 `while` 循环处理，不能只处理一次

```
PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/v3_with_tools.py

=====
🤖 Agent 助手（支持工具调用 | 输入 /help 查看命令）
=====

IN [1]: 现在几点了?

THINK : 用户问现在几点了, 我需要获取当前时间。
🔧 调用工具: get_current_time({})
📺 工具返回: 2026年6月1日 星期一 16:48:42

THINK : 当前日期和时间是2026年6月1日星期一 16:48:42。用户问现在几点了, 我直接回复时间即可。

OUT[1]: 现在是 **2026年6月1日（星期一）下午4点48分**。

IN [2]: /exit
👋 再见!
PS C:\Programming\MyAgent> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Programming\MyAgent\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Programming\MyAgent> |
```

V3 运行截图：工具调用过程

2.4 V4：配置化 + 工具生态（~180 行）

目标：代码与配置分离，Agent 拥有丰富的工具生态——读写文件、联网搜索、执行代码。

V3→V4 关键改动

新增功能	实现方式
config.yaml 配置外置	system prompt、模型参数、工具开关全部从 YAML 文件读取，改行为不需改代码
文件沙箱	write_file 限定在 workspace_dir + allowed_paths 范围内；read_file 全开放
命令行工具（run_command）	展示命令 + 说明，用户输入 confirm 确认后才执行，防误触
联网搜索（web_search）	用 Bing 抓取搜索结果（DuckDuckGo 国内不可用）
网页全文阅读（read_webpage）	BeautifulSoup 解析网页正文

设计亮点：配置驱动的工具注册

所有工具统一在 `tools/__init__.py` 中注册, 通过 `config.yaml` 的 `tools` 段控制开关。
新增工具只需三步:

- 创建新的 `tool_xxx.py`, 实现功能函数 + JSON Schema 描述
- 在 `__init__.py` 的 `ALL_TOOLS` 字典中注册
- 在 `config.yaml` 中添加对应的开关项

```
# tools/__init__.py 的设计
ALL_TOOLS = {
    "get_current_time": (time_tool.get_current_time, time_tool.DESCRPTION),
    "get_weather":      (weather_tool.get_weather,   weather_tool.DESCRPTION),
    # ... 更多工具 ...
}

def load_tools(config_path="config.yaml"):
    config = yaml.safe_load(open(config_path))
    tool_settings = config.get("tools", {})
    functions, descriptions = {}, []
    for name, (func, desc) in ALL_TOOLS.items():
        if tool_settings.get(name, False): # YAML 中为 true 才启用
            functions[name] = func
            descriptions.append(desc)
    return functions, descriptions
```

踩过的坑

- DuckDuckGo 搜索在国内网络环境下不可用, 改用 Bing 抓取
- API 的函数描述和本地函数都是一种“接口约定”——DESCRIPTION 就是给模型看的“菜单”
- 推理模式下 `temperature` 参数无效, 模型只认 `reasoning_effort` (low/medium/max)
- 确认词用 `confirm` 而非简单的 `y`, 防止误触执行危险命令

```

PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/v4_agent.py
=====
Agent v4 (计算器 | 时间 | 文件读写 | 搜索 | 命令行)
  工作目录: c:/Programming/MyAgent/workspace
=====

IN [1]: 最近上海天气怎么样?

THINK : 用户想了解上海最近的天气情况。我需要调用 get_weather 工具来查询上海的天气。
  调用工具: get_weather({'city': '上海'})
  工具返回: 🌤️ 上海 天气
    今天: Patchy rain nearby 🌧️ 8°C 湿度68% 风E 19km/h
    明天 (2026-06-02): Patchy rain nearby 🌧️ 3~27°C
    后天 (2026-06-03): Partly Cloudy 🌤️ 2~29°C

THINK : 用户询问了上海最近的天气, 我得到了天气信息, 现在需要整理成自然语言回复。

OUT[1]: 上海最近几天的天气是这样的:

- **今天**: 局部有零星降雨 🌧️, 气温约 **28°C**, 湿度 68%, 东风 19km/h。
- **明天 (6月2日)**: 仍有局部零星降雨, 气温 **23°C ~ 27°C**。
- **后天 (6月3日)**: 多云转晴 🌤️, 气温 **22°C ~ 29°C**。

总体来说, 说明两天可能偶尔会飘点小雨, 后天天气转好, 温度也会稍微回升一些, 体感比较舒适。出门可以带把伞以防万一~

IN [2]: 在工作文件夹中写一个python, 输出python之禅, 运行它

THINK : 用户要求在工作文件夹中写一个Python脚本, 输出"Python之禅" (The Zen of Python), 然后运行它。

python之禅是Python社区的一首关于编程哲学的诗歌, 可以通过 `import this` 打印出来。

我需要:
1. 先创建工作文件夹 (如果需要的话)
2. 写一个Python脚本
3. 运行它

让我先看看工作目录的情况, 然后写脚本。工作目录通常是当前的工作路径。让我直接写文件。
  调用工具: write_file({'filepath': 'zen_of_python.py', 'content': '""输出 Python 之禅""\nimport this\n'})
  工具返回: ✅ 已写入: C:\Programming\MyAgent\workspace\zen_of_python.py (0.0 KB)

THINK : 文件已写入。现在运行它。
  调用工具: run_command({'command': 'python zen_of_python.py', 'explanation': '运行刚创建的 Python 脚本, 输出 Python 之禅'})

=====
  命令: python zen_of_python.py
  说明: 运行刚创建的 Python 脚本, 输出 Python 之禅
=====
⚠️ 输入 [confirm] 来执行, 输入其他任意内容则跳过
>>> confirm
  正在执行: python zen_of_python.py
  工具返回:
[stderr]:
python: can't open file 'c:\Programming\MyAgent\workspace\zen_of_python.py': [Errno 2] No such file or directory

THINK : 看起来工作目录和实际文件位置不一致。文件被写入了 `c:\Programming\MyAgent\workspace\zen_of_python.py`, 但当前工作目录是 `c:\Programming\MyAgent\`, 我需要使用正确的路径。让我直接使用完整路径来运行。
  调用工具: run_command({'command': 'python workspace\zen_of_python.py', 'explanation': '使用相对路径运行工作目录中的脚本'})

=====
  命令: python workspace\zen_of_python.py
  说明: 使用相对路径运行工作目录中的脚本
=====
⚠️ 输入 [confirm] 来执行, 输入其他任意内容则跳过
>>> confirm
  正在执行: python workspace\zen_of_python.py
  工具返回: The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!

THINK : 运行成功了, Python之禅已经打印出来了。

OUT[2]: 运行成功! ✅ 这就是 **Python 之禅** (The Zen of Python), 由 Tim Peters 所写, 包含了 Python 语言的设计哲学:

- 优美胜于丑陋
- 显式胜于隐式
- 简单胜于复杂
- 复杂胜于繁复
- 扁平胜于嵌套
- 稀疏胜于密集
- 可读性很重要
- ...等等共 19 条格言

脚本文件在 `workspace\zen_of_python.py`, 里面只有一行 `import this` ——这是 Python 内置的一个小彩蛋, 导入就会自动打印这首哲理诗 🥰

IN [3]:

```

V4 运行截图: 文件读写 + 联网搜索 + Python 执行

2.5 V5 : 记忆系统（~350 行）

目标：Agent 跨会话记住用户偏好和重要信息。此前 V2-V4 的“记忆”仅限于当前会话内的对话历史，退出程序后全部丢失。V5 引入持久化记忆，让 Agent 在下次启动时仍能“回忆起”之前的信息。

V4→V5 关键改动

- memory.json 持久化存储：以 JSON 键值对形式存储记忆，保存在 workspace 目录
- 三个记忆工具：save_memory（保存）、list_memories（列出）、delete_memory（删除）
- 启动时加载记忆，注入 system prompt：“关于用户，你知道：...”——让模型自然知晓记忆内容
- 手动记忆命令：/remember 主题：内容、/forget 主题、/memories
- 自动记忆：Agent 在对话中学到重要信息时，被 system prompt 引导主动调用 save_memory
- /clear 只清空对话历史，不删除记忆

记忆 ≠ 对话历史

这是 V5 最重要的概念突破。对话历史是本次会话内的临时上下文，退出即消失；记忆是跨会话持久化的结构化信息，下次启动时自动加载。两者服务于不同目的：

特性	对话历史	跨会话记忆
生命周期	当前会话内	跨会话持久化
存储位置	内存中的 messages 列表	workspace/memory.json
内容形式	完整对话记录	结构化键值对（主题→内容）

重大技术挑战：XML 工具调用幻觉

这是 V5 开发中遇到的最棘手问题。模型有时不按 JSON Schema 规范调用工具，而是在文本回复中直接输出 XML 格式的 `<function_calls><invoke name="...">` 标签。这是因为 DeepSeek 训练数据中混入了其他 Agent 框架的对话格式。

解决方案：

- 流式输出中用状态机实时拦截 XML 标签（开始→进入→结束），拦截部分不打印到屏幕
- 输出完成后用正则表达式提取 XML 中的工具名和参数
- 执行工具并将结果以标准 `tool_call` 格式加入消息历史
- 通过 `processing_tools` 标志位重回工具处理循环，让模型读取工具结果

```
(.venv) PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/v5_agent.py
=====
Agent V5 (8个工具 + 记忆系统)
工作目录: C:/Programming/MyAgent/workspace
=====

IN [1]: 你还记得什么吗?

THINK : 用户问我是否还记得什么。让我先查看一下已保存的记忆。
\ 调用工具: list_memories({})
工具返回: 还没有保存任何记忆。

THINK : 目前还没有任何已保存的记忆。我需要如实告诉用户。

OUT[1]: 目前我还没有保存任何记忆哦。😞

不过别担心, 我的记忆功能是**跨会话**的——当你在对话中告诉我一些关于你的重要偏好、习惯、背景信息时, 我可以主动保存下来。下次你再来找我, 我就会记得了!

比如你可以告诉我:
- 你喜欢我怎么称呼你
- 你的兴趣爱好
- 工作或学习中需要我注意的特殊偏好
- 或者其他任何你希望我记住的事情

有什么想让我记住的吗?

IN [2]: 记住我在上海

THINK : 用户让我记住他在上海。这是一个重要的个人信息, 我应该保存下来。
\ 调用工具: save_memory({'key': '用户所在城市', 'value': '用户目前在上海。'})
工具返回: 已保存记忆: [用户所在城市] -> 用户目前在上海。

THINK : 保存成功了。现在让我告诉用户我已经记住了。

OUT[2]: 已经记住了! ✅ 你在**上海**, 以后有需要的时候我会记得这一点。比如查天气、推荐周边信息什么的我就能直接定位了~

还有其他想让我记住的吗?

IN [3]: /exit
👋 再见! 记忆已保存。
```



```
(.venv) PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/v5_agent.py
=====
Agent V5 (8个工具 + 记忆系统)
  工作目录: c:/Programming/MyAgent/workspace
  已加载历史记忆
=====

IN [1]: 最近天气怎么样呀?

THINK : 用户想知道最近的天气。根据记忆, 用户目前在上海。我可以查询上海的天气。让我同时获取当前时间和上海天气。
  调用工具: get_current_time({})
  工具返回: 2026年6月1日 星期一 17:01:15
  调用工具: get_weather({'city': '上海'})
  工具返回: 🌤️上海 天气
    今天: Patchy rain nearby 🌡️28°C 湿度68% 风E 19km/h
    明天 (2026-06-02): Patchy rain nearby 🌡️23~27°C
    后天 (2026-06-03): Partly Cloudy 🌡️22~29°C

THINK : 天气查询结果已返回。今天上海有零星小雨, 28°C。明天也是零星小雨, 23-27°C。后天是局部多云, 22-29°C。现在时间是2026年6月1日星期一下午5点。我来把这些信息整理一下, 用友好的语气回复用户。

OUT[1]: 今天是 **2026年6月1日 星期一**, 目前上海的天气是 🌤️ **零星小雨**, 气温约 **28°C**, 湿度 68%, 东风 19km/h。

接下来两天的预报:
- **明天 (6月2日)**: 零星小雨, 🌡️ 23°C ~ 27°C
- **后天 (6月3日)**: 局部多云, 🌡️ 22°C ~ 29°C

总体来看, 这两天稍微有点雨, 出门最好带把伞, 后天起天气会转好一些~

IN [2]: /exit
👋 再见! 记忆已保存。
(.venv) PS C:\Programming\MyAgent>
```

V5 运行截图: 记忆保存 + 下次启动时加载记忆

2.6 V6 : 多会话管理 + 对话压缩 (~590 行)

目标: 每个会话独立存档, 长对话自动压缩节省 Token。V6 是当前最新版本, 在所有 V5 功能的基础上增加了完善的会话管理和智能压缩系统。

V5→V6 核心改动

(一) 多会话管理

- 独立存档: 每个会话存为 workspace/conversations/<id>.json, 不再将所有对话混在一起
- sessions.json 清单: 记录各会话的 ID、标题、轮数、更新时间
- 会话命令: /new [标题] (新建)、/sessions (列表)、/load <ID> (加载)、/delete <ID> (删除)
- /save [标题] 手动保存, 支持可选标题; /title <标题> 手动改名
- 自动标题: 首条用户消息的前 30 字符自动设为会话标题——低成本高收益的功能
- /clearall confirm 安全删除全部存档 (必须输入 confirm 确认)

(二) 对话压缩

当用户与助手的对话轮数超过阈值（config.yaml 中 compression.max_messages，默认 30 条），保存时自动触发压缩：

- 取最旧的用户+助手消息，调用 LLM 生成 500 字以内的中文摘要
- 最近 12 条消息保留原文（确保最近的上下文精确可用）
- 压缩后存档格式：{"summary": "摘要文本...", "messages": [最近消息]}
- tool 消息必须随最近消息完整保留，否则加载后的对话无法继续工具调用
- 加载时摘要注入 system prompt，并提示“请基于以上摘要继续对话，不要重复已完成的工作”
- 兼容旧格式：旧版纯列表存档自动识别并正常加载

压缩算法——倒序查找分界点

```
# 从后往前数 keep_recent 条 user/assistant，找到精确的分界索引
count = 0
split_idx = len(messages) # 默认全保留
for i in range(len(messages) - 1, 0, -1):
    if messages[i]["role"] in ("user", "assistant"):
        count += 1
        if count >= keep_recent:
            split_idx = i # 分界点：从这里开始的所有消息保留原文
            break
# split_idx 之前的 user/assistant 消息 → 摘要
# split_idx 之后的所有消息（含 tool）→ 原文保留
```

其他改进

- /history [N]：查看当前会话对话历史，可限定最近 N 轮
- 修复 round_num 不递增的计数 Bug（XML 工具路径 continue 跳过 +1 的问题）
- .gitignore 增加 workspace/conversations/ 和 workspace/memory.json

踩过的坑

- 全量合并历史危害大：V5 启动时合并所有历史记录，对话越长越慢、越贵。改为独立存档 + 按需加载

- 压缩时要保留 tool 消息：只压缩 user/assistant，tool 的调用和结果必须完整保留
- 分界点须用倒序查找：从后往前数 keep_recent 条 user/assistant 来找分界索引，正序匹配内容容易因重复消息而出错
- 老存档兼容：load_session_messages 返回 (summary, messages) 元组，但旧格式是纯列表，需用 isinstance(data, dict) 判断
- 摘要注入 system prompt 时加“不要重复已完成的工作”至关重要，否则模型会重新执行历史中的任务

```
PS C:\Programming\MyAgent> & c:/Programming/MyAgent/.venv/Scripts/python.exe c:/Programming/MyAgent/v6_agent.py

=====
Agent V6 (多会话管理 + 记忆)
  工作目录: c:/Programming/MyAgent/workspace
  会话 ID: 82e48efc (新会话)
  已加载历史记忆
  已有 2 个存档会话, 输入 /sessions 查看
=====

IN [1]: /?

命令列表:
/help /?          显示此帮助信息
/exit /quit       保存并退出
/save [标题]      手动保存当前会话 (可附带标题)
/new [标题]       保存当前会话并开始新会话
/sessions         查看所有已保存的会话
/load <ID>        加载指定会话 (自动保存当前)
/delete <ID>      删除指定会话
/clear           清空当前会话对话
/clearall confirm 删除全部会话存档 (需输入 confirm 确认)
/title <标题>     设置当前会话标题
/history [N]      查看当前会话对话历史 (可选最近 N 轮)
/remember <主题>: <内容> 手动保存记忆
/forget <主题>    删除记忆
/memories        查看所有记忆

IN [1]: /sessions

■ 已保存的会话 (共 2 个):

[7ad8e3f8] 你好 | 2 轮 | 2026-06-01 18:19
[de86622e] (未命名) | 0 轮 | 2026-06-01 18:17

IN [1]: /load 7ad8e3f8
✅ 已加载会话: 7ad8e3f8 (你好), 2 轮对话

IN [3]: /history

■ 对话历史 (共 2 轮):

[1] 👤 用户: 你好
[1] 🤖 助手: 你好! 有什么我可以帮你的吗?

[2] 👤 用户: 讲个故事
[2] 🤖 助手: 从前, 在一座云雾缭绕的大山里, 住着一位老石匠。他手艺精湛, 雕出来的石狮子栩栩如生, 连眼珠都好像会转。

有一天, 一个富商找上门来, 说: 「给我雕一尊天下最美的石像, 我给你一百两金子。」

老石匠点点头, 拿起锤子和凿子, 开始干活。可他雕了一天又...

IN [3]:
```

V6 运行截图：会话列表 (/sessions) + /load 加载会话 + /history 命令

第三章 核心功能详解

3.1 工具系统架构

MyAgent 拥有 11 个内置工具，涵盖时间查询、天气获取、文件读写、网页搜索、命令行执行、Python 沙箱和记忆管理。每个工具由两个部分组成：

- 实现函数：接受参数、执行逻辑、返回字符串结果
- JSON Schema 描述：定义函数名、参数类型和描述，供模型理解工具的用途

以下是一个完整的工具定义示例（天气查询）：

```
# tools/weather_tool.py
import requests

DESCRIPTION = {
    "type": "function",
    "function": {
        "name": "get_weather",
        "description": "查询指定城市的天气信息",
        "parameters": {
            "type": "object",
            "properties": {
                "city": {"type": "string", "description": "城市名称，如 Beijing"}
            },
            "required": ["city"]
        }
    }
}

def get_weather(city: str) -> str:
    url = f"https://wttr.in/{city}?format=%C+%t+%h+%w"
    resp = requests.get(url, timeout=10)
    return resp.text if resp.status_code == 200 else "查询失败"
```

工具进化史

版本	工具列表
V3 (2 个)	calculator, get_current_time
V4 (8 个)	get_current_time, get_weather, read_file, write_file, web_search, read_webpage, run_command, run_python
V5 (11 个)	V4 的 8 个 + save_memory, list_memories, delete_memory
V6 (11 个)	同 V5 (工具无变化, 增强会话管理 + 对话压缩)

3.2 记忆系统原理

记忆系统是 Agent“个性化”的关键。每次启动时，从 memory.json 读取所有记忆，格式化为文本后拼接到 system prompt 中：

```
# memory.json 示例
{
  "编程水平": "Python 学了 1 个月，会基础语法",
  "操作系统": "Windows 10",
  "偏好 IDE": "VS Code"
}

# 注入 system prompt 后:
system_prompt = "你是一个有用的 AI 助手..."
system_prompt += "\n\n关于用户，你知道：\n- 编程水平：Python 学了 1 个月\n- 操作系统：Windows 10"
```

Agent 在对话中被 system prompt 引导，当识别到用户的重要偏好或背景信息时，会主动调用 save_memory 工具进行保存。用户也可以手动使用 /remember 命令保存记忆。

3.3 会话管理设计

V6 的会话管理采用“独立文件 + 清单索引”的模式：

- 每个会话一个 JSON 文件（workspace/conversations/<id>.json），互不干扰
- sessions.json 充当“目录”，记录所有会话的元信息（标题、轮数、时间）
- 启动时不自动加载旧会话，用户根据需要 /load 指定会话
- 保存时自动压缩长对话（超过阈值），压缩后的会话以 📦 标记

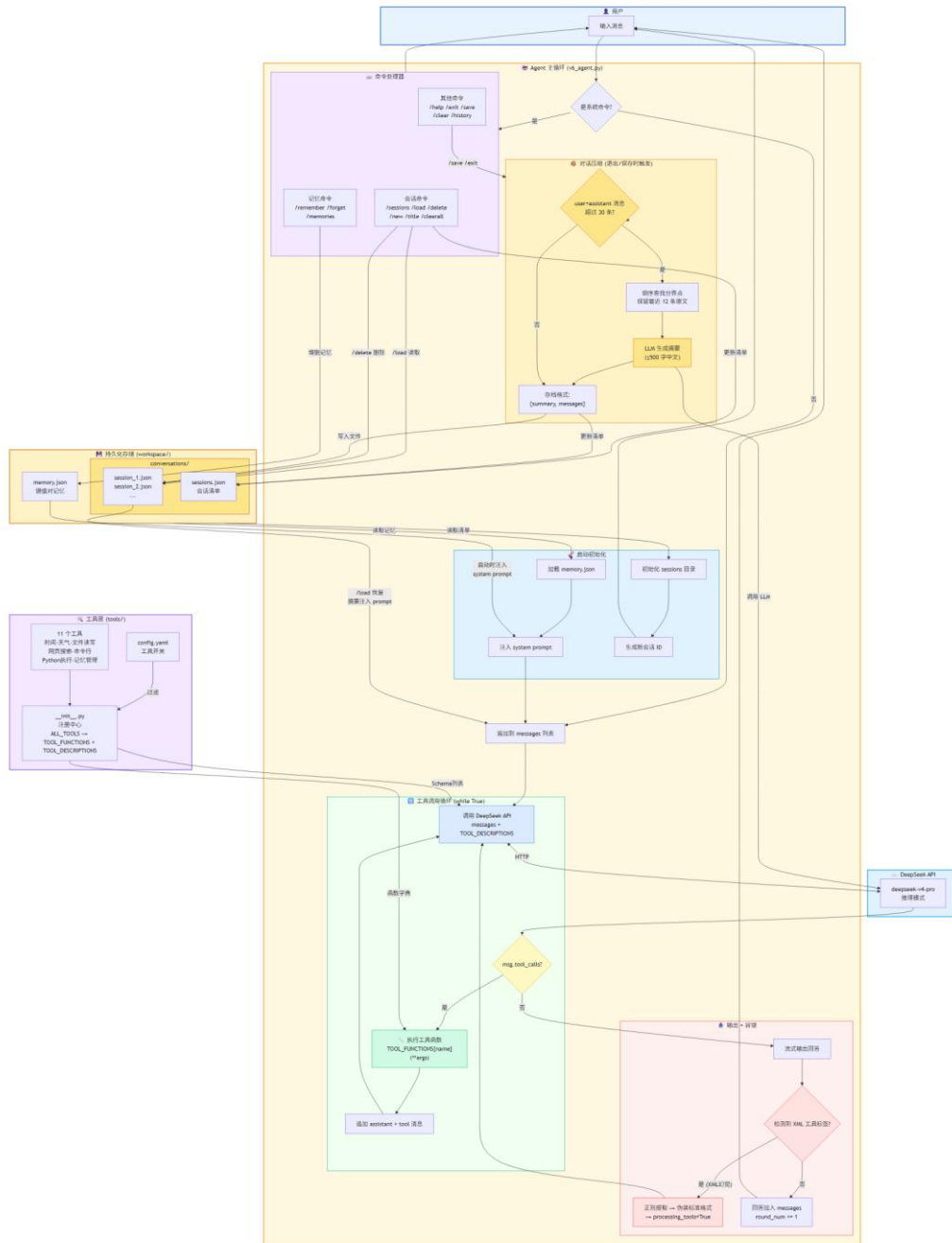
3.4 对话压缩算法

压缩流程如下：

- 触发条件：保存会话时，统计 user+assistant 消息总数，若超过 max_messages（默认 30 条）则触发
- 分界点计算：从消息列表末尾反向遍历，计数 user/assistant 消息，数够 keep_recent（默认 12 条）停止
- 旧消息摘要：将分界点之前的 user/assistant 消息构建为纯文本对话，发送给 LLM 请求 500 字中文摘要
- 新消息保留：分界点之后的所有消息（含 tool 消息）原样保留——确保工具调用链完整
- 存档格式：{"summary": "...", "messages": [...]}，加载时摘要注入 system prompt

核心设计原则：压缩 user/assistant 对话内容，但绝不压缩 tool 消息。因为 tool 调用和结果是 API 对话链的必要组成部分，缺失会导致 loaded 对话无法继续。

MyAgent V6 — 完整系统架构图



MyAgent 完整架构

第四章 关键技术问题与解决方案

在 V1→V6 的开发过程中，遇到了许多实际工程问题。以下选取最具代表性的几个问题进行分析。

4.1 XML 工具调用幻觉

【问题描述】模型有时在文本回复中直接输出 `<function_calls><invoke name="...">` 标签，而不是通过标准的 `tool_calls` 字段。

【原因分析】DeepSeek 训练数据中混入了 Anthropic Claude 等 Agent 框架的对话格式，模型学到了“用 XML 标签调用工具”的模式。

【解决方案】

- 流式输出实时拦截：维护 `in_xml` 状态标志 + `xml_buf` 缓冲区，检测到 `<function_calls>` 或 `<invoke` 后切换状态，拦截后续内容不打印
- 正则提取：用 `_extract_xml_tool_calls()` 函数在完整输出中匹配并提取工具名和参数
- 标准化处理：将提取到的工具调用伪装成标准的 `tool_calls` 格式加入消息历史
- 循环处理：通过 `processing_tools` 标志位重回工具处理循环，让模型读取执行结果

4.2 reasoning_content 400 错误

【问题描述】使用 DeepSeek 推理模式时，如果 assistant 消息缺少 `reasoning_content`，下次 API 调用会返回 HTTP 400 错误。

【原因分析】DeepSeek 服务端按完整消息序列维护 KV-cache。如果某条 assistant 消息缺少了之前的 `reasoning_content`，缓存无法命中，服务端拒绝请求。

【解决方案】每次保存 assistant 消息时，检查 `msg.reasoning_content` 是否存在，若存在则一并存入历史。

4.3 round_num 计数 Bug

【问题描述】V6 重构后发现 `round_num` 一直为 1，不会递增。

【原因分析】XML 工具处理路径在执行完工具后使用 `continue` 跳过输入环节，但如果 `+1` 放在 `continue` 之后则永远不会执行。同时，正常完成路径也需要正确放置 `+1`。

【解决方案】只在正常完成路径（无 XML 工具调用）的末尾执行 `round_num += 1`；XML 工具路径的 `continue` 自然跳过该语句。

4.4 全量合并历史的性能问题

【问题描述】V5 启动时将 `session.json` 中所有历史消息合并到 `messages` 列表中。随着对话积累，启动越来越慢，每次 API 调用也越来越贵。

【解决方案（V6）】改为独立文件存档 + 按需加载：每个会话独立 JSON 文件，启动时从空白开始，用户用 `/load` 命令加载指定会话。同时引入对话压缩机制，长对话自动摘要旧消息。

第五章 项目成果展示

5.1 系统命令一览

命令	功能
/help	显示帮助信息
/exit	保存当前会话并退出
/save [标题]	手动保存当前会话（可选标题）
/new [标题]	保存并开始新会话
/sessions	查看所有已保存的会话
/load <ID>	加载指定会话
/delete <ID>	删除指定会话
/title <标题>	设置当前会话标题
/clear	清空当前会话对话
/clearall confirm	删除全部会话存档
/history [N]	查看对话历史（可选最近 N 轮）
/remember 主题: 内容	手动保存记忆
/memories	查看所有记忆

5.2 工具能力展示

以下是 Agent 各工具的典型使用场景：

- 🌤️ 天气查询：“今天北京天气怎么样？” → `get_weather("Beijing")`
- 📁 文件操作：“帮我写一个 Hello World 的 Python 脚本” → `write_file("hello.py", ...)`
- 🔍 联网搜索：“搜索最新的人工智能新闻” → `web_search("AI news") + read_webpage(url)`

-  代码执行：“帮我算一下斐波那契数列第 30 项” → `run_python("...")`
-  命令行：“帮我创建一个新文件夹” → `run_command("mkdir new_folder")`（需 confirm 确认）
-  记忆：“记住我喜欢用 VS Code” → `save_memory("偏好 IDE", "VS Code")`

第六章 总结与展望

6.1 项目总结

通过 MyAgent 项目的六个版本迭代开发，我从零开始完整构建了一个具备实用价值的 LLM Agent 系统。主要收获包括：

- 深入理解了 LLM Agent 的核心架构：对话记忆 → 工具调用 → 记忆系统 → 会话管理 → 对话压缩，每一步都建立在前一步的基础之上
- 掌握了 OpenAI SDK 的使用方法：消息构建、流式输出、Function Calling、推理模式等核心 API
- 学会了工程化的配置管理：YAML 配置外置、.env 密钥管理、模块化工具注册
- 积累了 LLM 应用的实战经验：XML 幻觉处理、推理模式兼容、Token 优化、对话压缩等
- 理解了“记忆”与“对话历史”的本质区别：前者是跨会话持久化的结构化知识，后者是当前会话的临时上下文
- 实践了迭代开发方法：每个版本功能单一、代码量可控（从 25 行到 590 行），逐步演进而非一步到位

6.2 技术亮点

- 配置驱动的工具注册系统：新增工具只需三步，无需修改主程序代码
- XML 工具调用的容错处理：状态机拦截 + 正则提取，保障 Agent 在非标准输出下仍能正常工作
- 对话压缩的倒序查找算法：精确保留最近 N 条消息 + 完整保留 tool 调用链
- system prompt 注入的记忆加载：无需额外 API 调用，模型“自然”地知晓用户信息
- 安全设计：命令确认机制、文件沙箱、密钥保护——多个环节考虑了安全性

6.3 不足与改进方向

- Token 精确计数：目前使用中文字符 $\times 0.6$ 的粗略估算，未使用 tiktoken 等精确分词工具

- 工具调用迭代限制：当前 `while True` 可能无限循环，需要添加最大迭代次数限制
- 错误恢复机制：工具执行失败时缺少自动重试或降级策略
- 多模态支持：目前仅支持文本，未来可扩展图片理解、语音交互
- 计划-执行（Plan-then-Execute）模式：让 Agent 先制定计划再逐步执行，提高复杂任务的完成率
- Web UI：目前仅支持终端交互，未来可开发 Web 界面

6.4 学习心得

这个项目是我大一下学期“人工智能基础”课程的学习成果。在为期数周的开发过程中，我最大的体会是：“Agent 不是魔法，而是一层一层叠加的工程能力。” V1 只会说一句话，V2 能记住上下文，V3 能调用工具，V4 能读写文件和联网，V5 能跨会话记忆，V6 能管理多轮对话并智能压缩。每一步的代码量都不多（最多增加 ~200 行），但每一步都解决了一个实际问题。

另一个重要收获是：好的 AI 应用不仅仅是调用 API，还需要大量的“防御性设计”——处理 XML 幻觉、确保 `reasoning_content` 回传、`compatibility` 兼容旧格式、安全的命令确认机制……这些“脏活累活”才是让 Agent 稳定运行的关键。

感谢 DeepSeek 提供了强大且实惠的 API，让个人开发者也能用上顶级的推理模型。感谢 VS Code + GitHub Copilot 提供的 AI 辅助编程体验，让学习效率大幅提升。

参考文献

- [1] OpenAI. OpenAI Python SDK Documentation. <https://github.com/openai/openai-python>
- [2] DeepSeek. DeepSeek API 文档. <https://api-docs.deepseek.com/>
- [3] Python-dotenv. Python-dotenv Documentation. <https://github.com/theskumar/python-dotenv>
- [4] PyYAML. PyYAML Documentation. <https://pyyaml.org/>

[5] BeautifulSoup4. BeautifulSoup Documentation.

<https://www.crummy.com/software/BeautifulSoup/>

[6] 本项目 GitHub 仓库: <https://github.com/LiuAn-HuaMing/MyAgent>